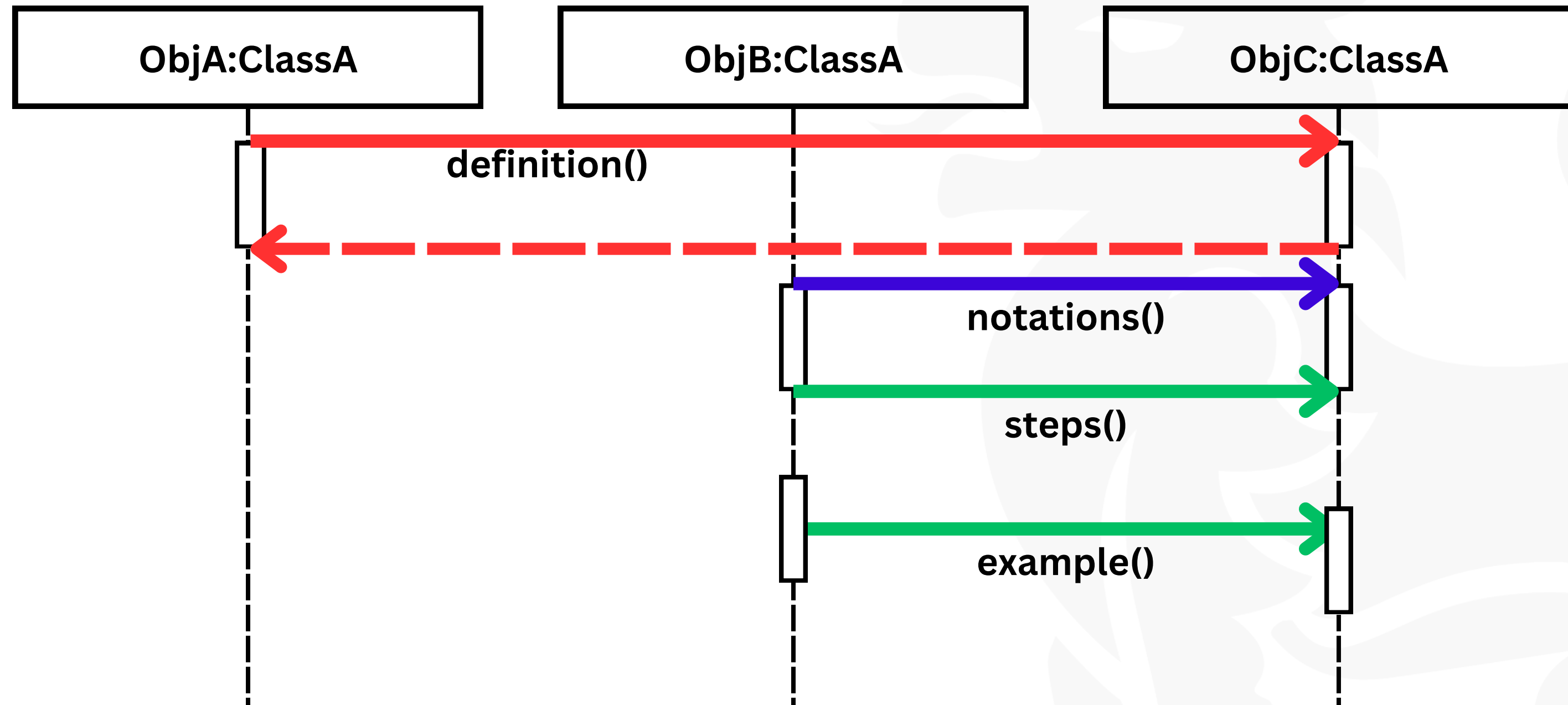


CMP5354

Software Design

UML Sequence Diagrams



UMLSequenceDiagramDefinition

A UML diagram that shows sequence of message (i.e. possible lines of code) between objects to carry out a specific use case or operation within a system.

UML Sequence Diagram Notations

Objects

varName:ClassA

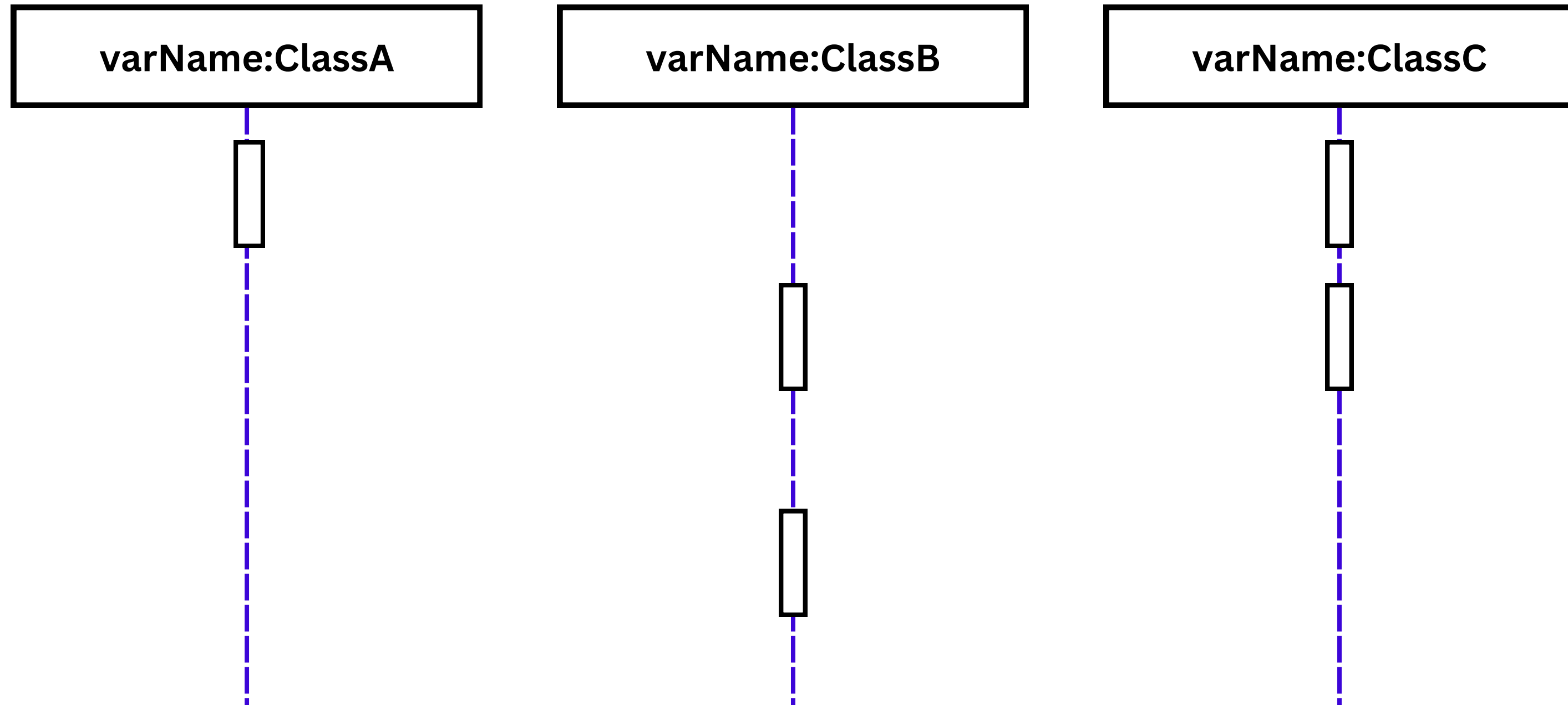
varName:ClassB

varName:ClassC

Objects are instances of the classes which will be created and involved in the execution of an operation/use case.

For example, to model a “user searches topic” use case, a User object is needed, and a NewsTopic object is needed.

Life Lines

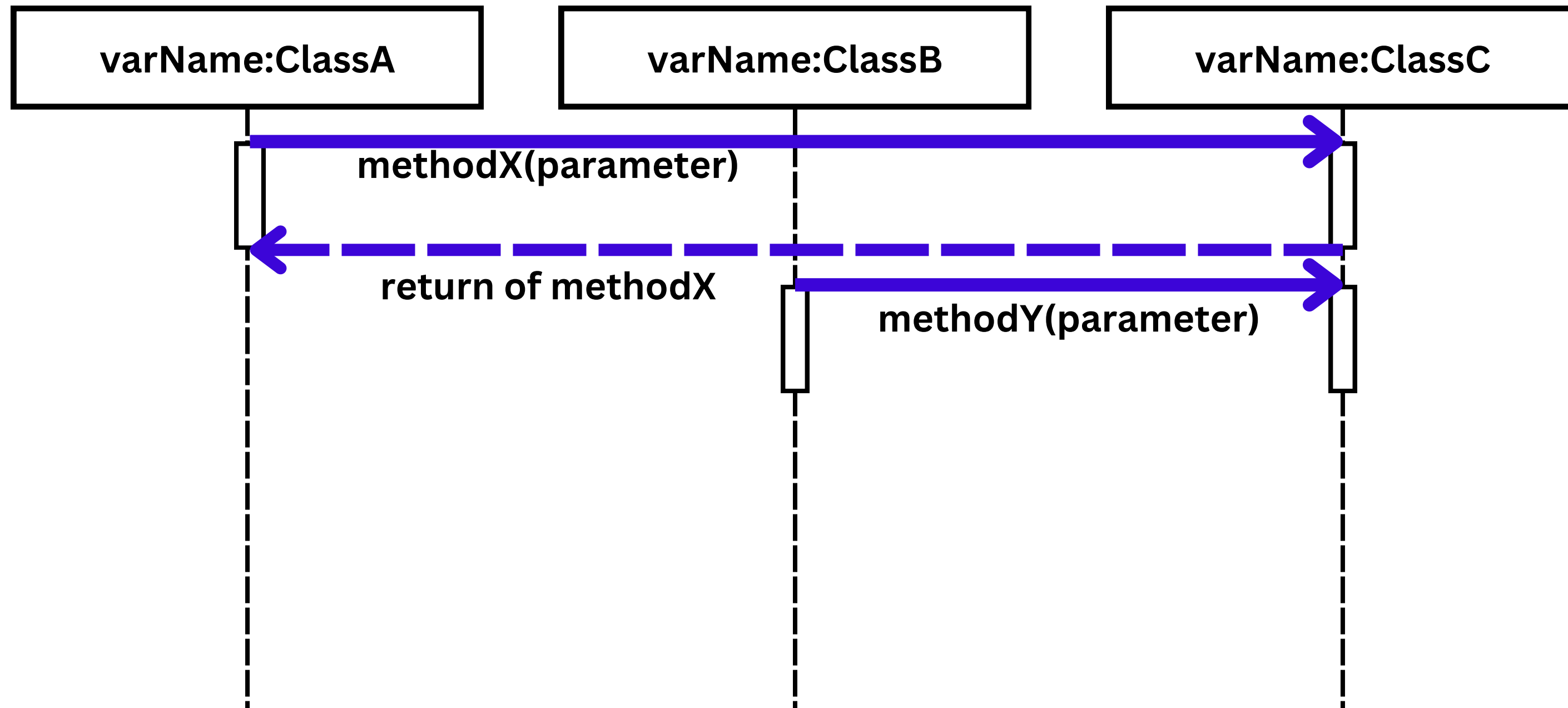


Life lines show the involvement of the object in the operation over time.

The dashed line indicates the object exists (not deleted).

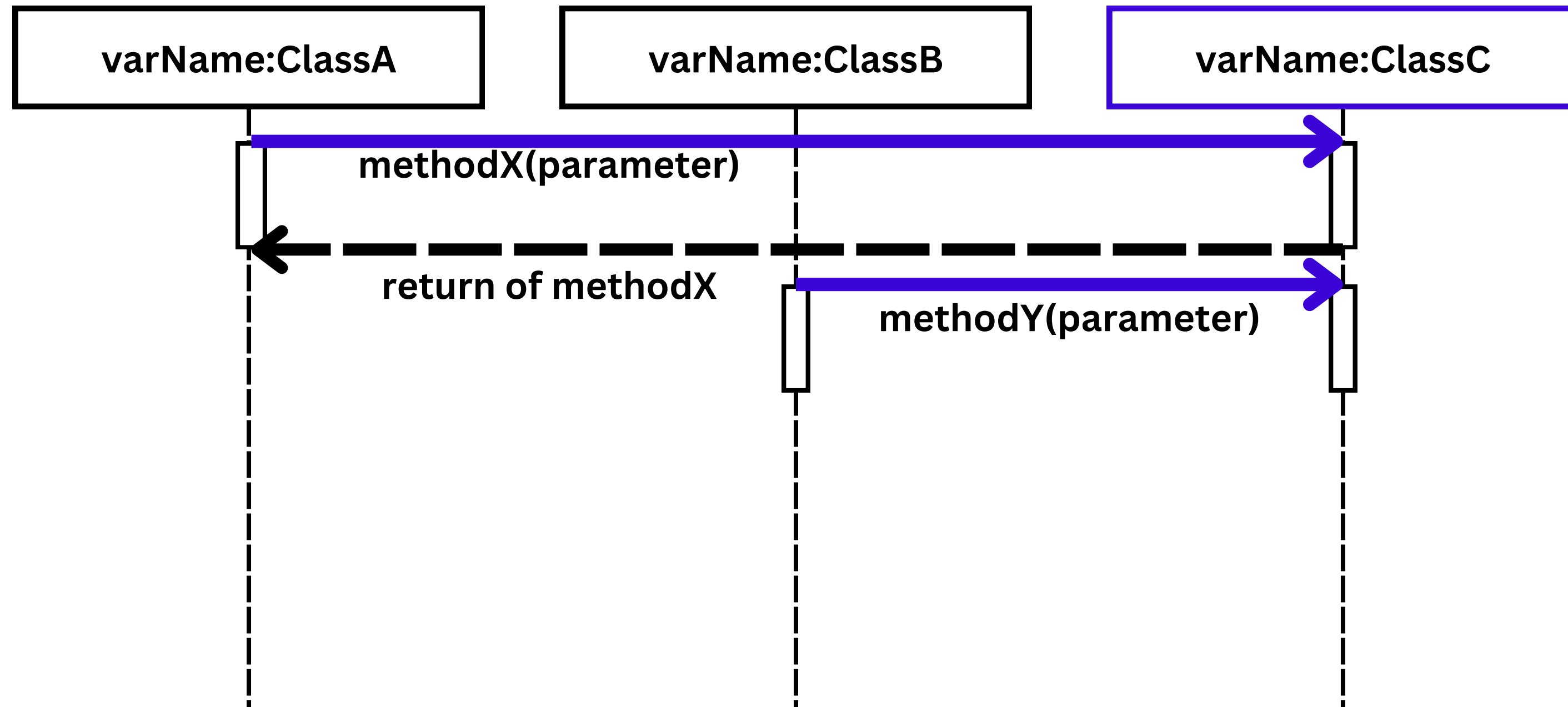
The rectangles show where the object is actively calling another one or being called.

Labelled Method Calls



Reading from top to bottom, the arrows should map to the expected steps within the modelled operations. The dashed arrow indicates that **methodX** should return a result. **MethodY** is “void” should does not have a dashed arrow in return.

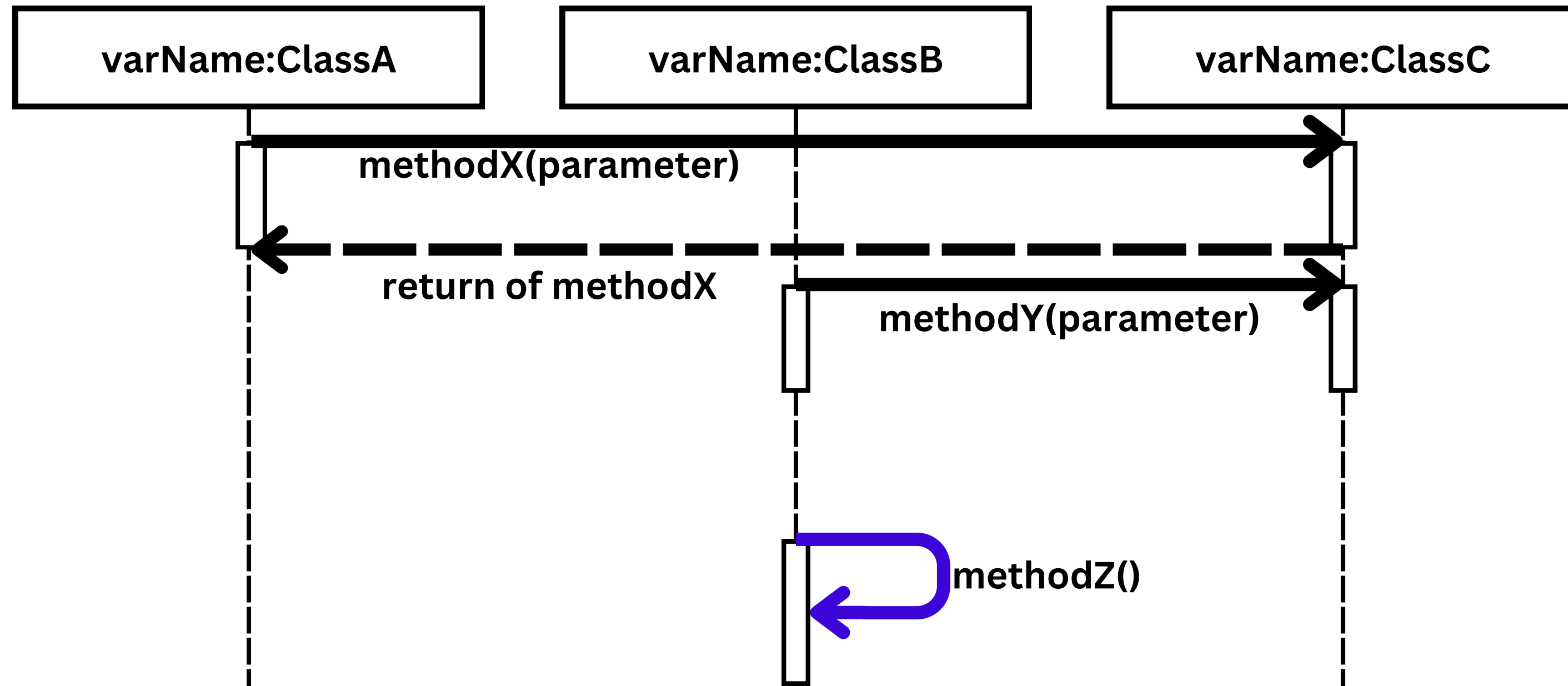
Labelled Method Calls



The direction of the solid arrows goes from the object calling the method towards the object where the method is implemented.

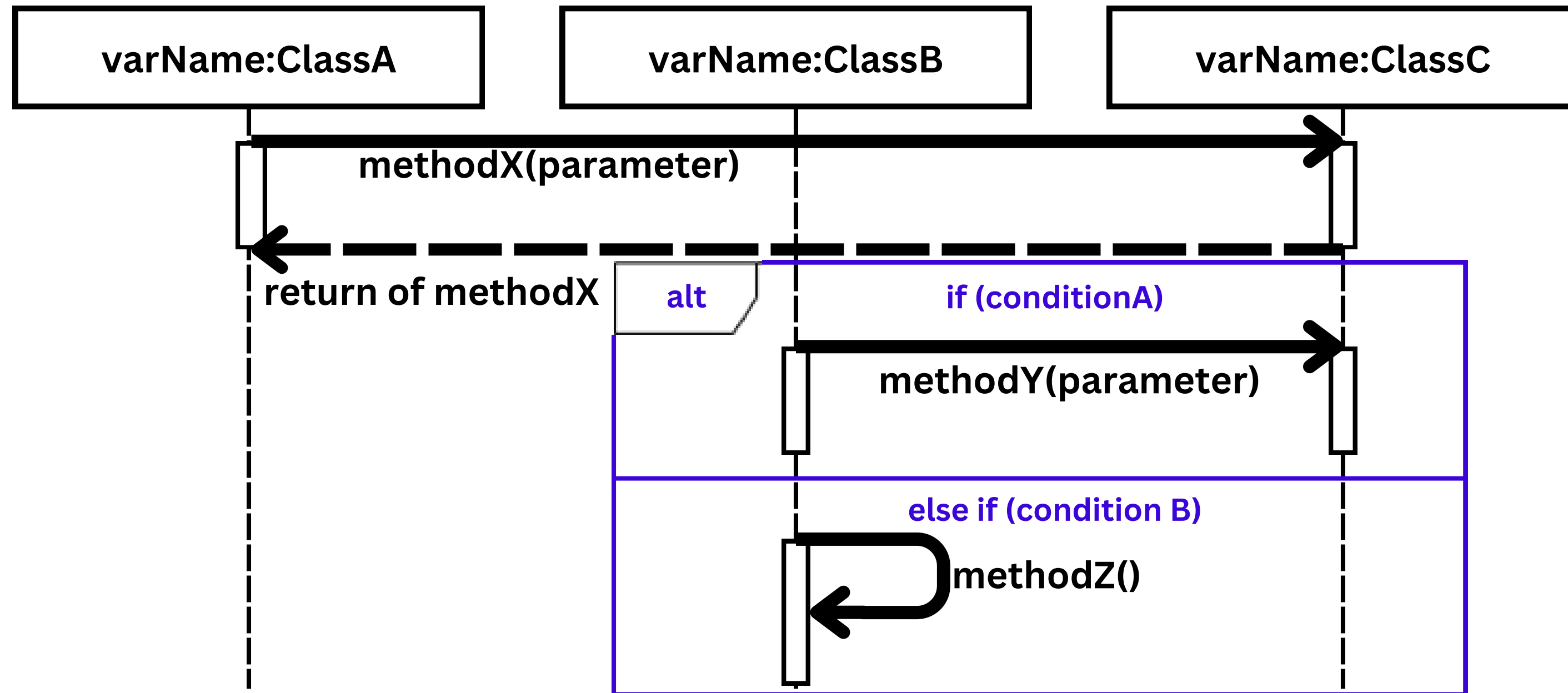
For examples, methodX and methodY should both be in ClassC in the class diagram.

Labelled Method Calls



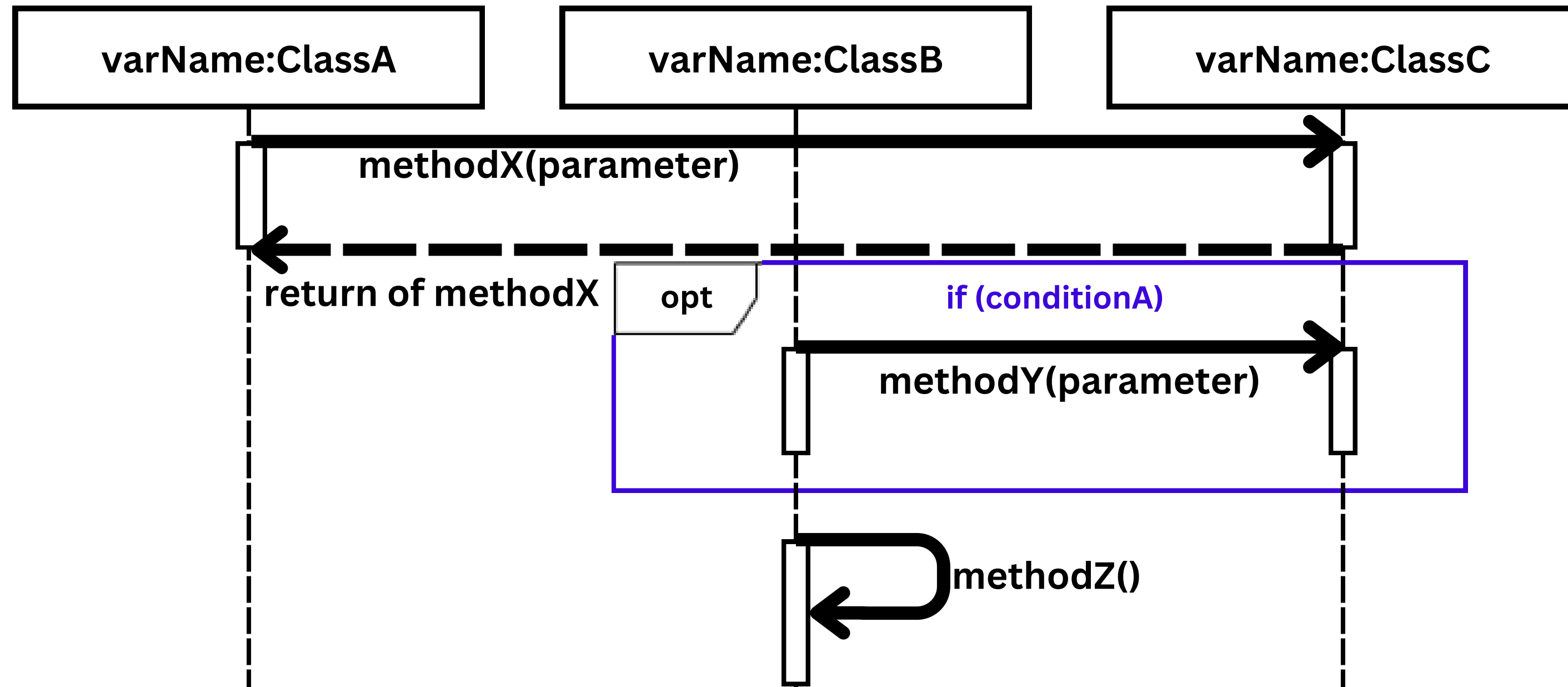
In some cases a method is implemented in a class and then called later in the same class as part of another method. For example, methodZ should be in ClassB in the class diagram. The sequence diagram then shows that methodZ would be called elsewhere in the same class.

Alt Envelopes



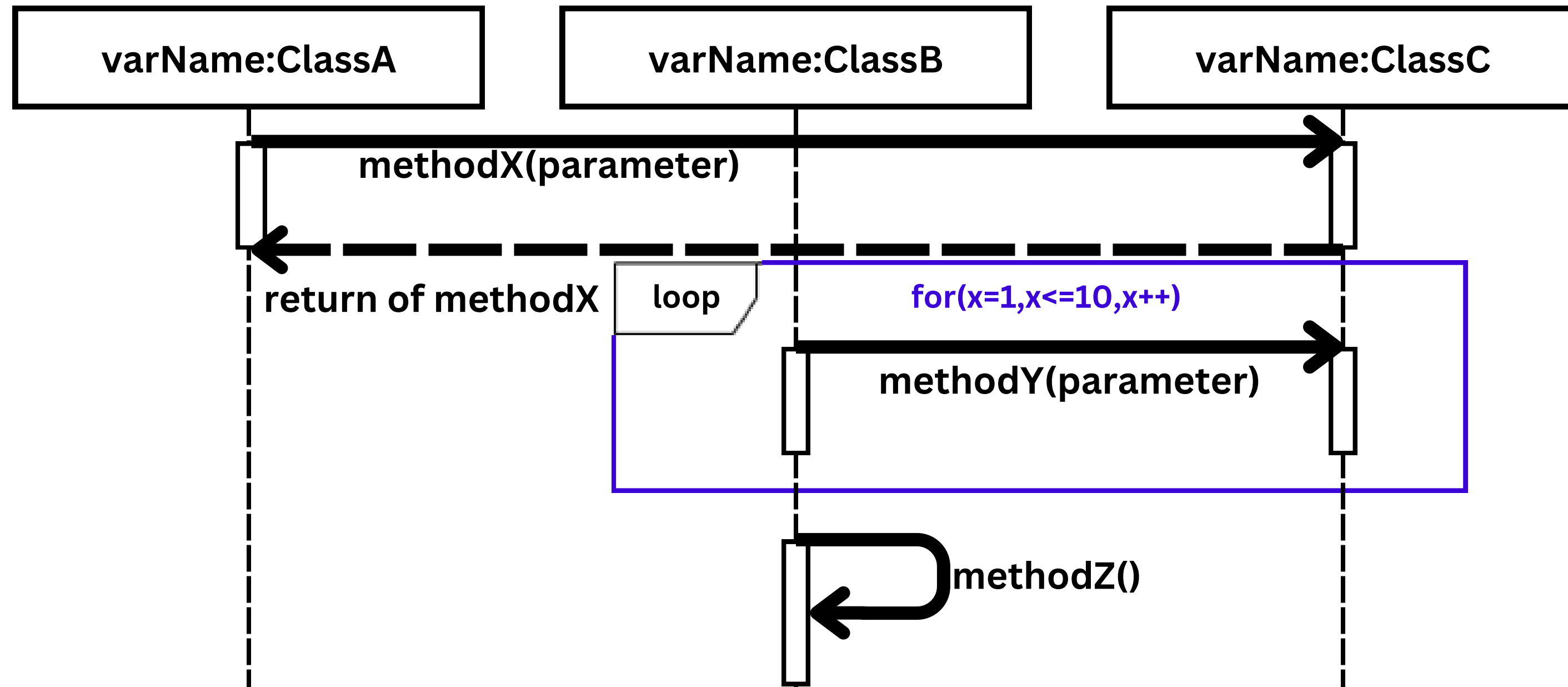
Alt (standing for “alternative”) envelopes are used to show if statements.
For example, if condition A is met, then methodY is called.
If condition B is met, methodZ is called.

Opt Envelopes



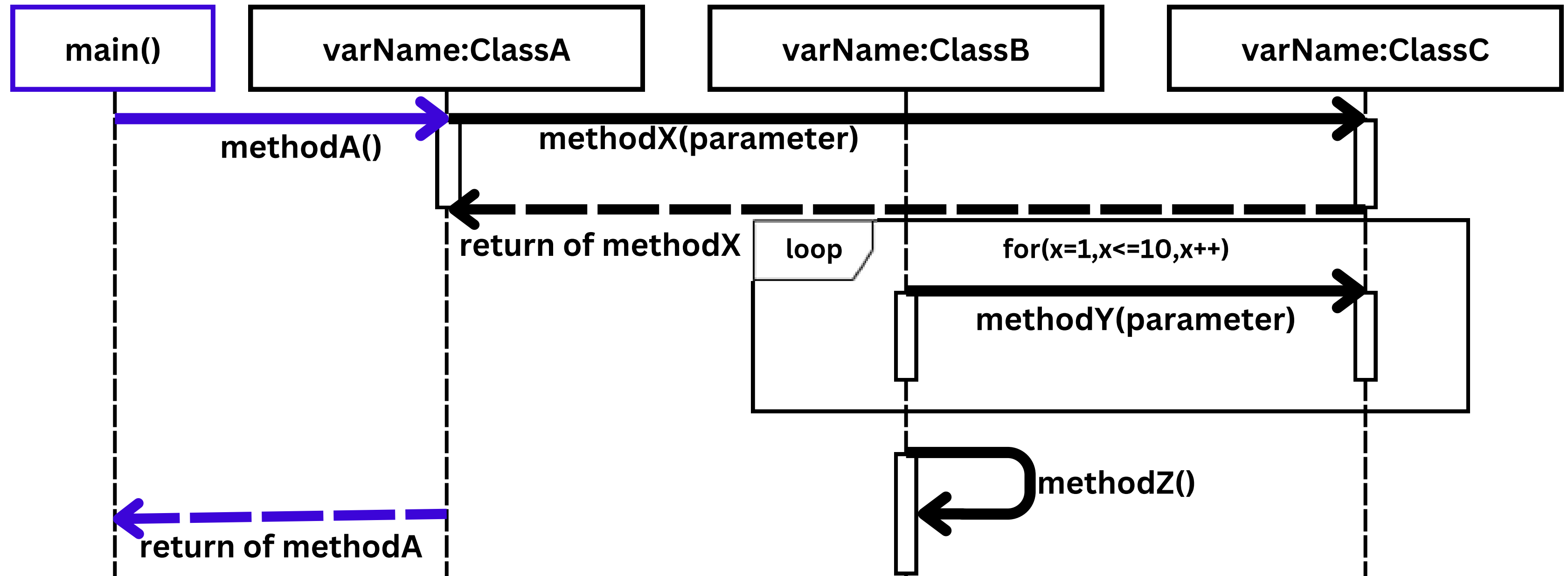
Opt (standing for “optional”) envelopes are used for if statements which do not have an else clause. For example, if condition A is met, then methodY is called, otherwise whatever is below the envelope is called.

Loop Envelopes



Loop envelopes are used to show for/while loops. For example, `methodY` is called 10 times before `methodZ` is called.

Main Trigger



Before the whole use case of the sequence diagram is triggered, a **main()** method (representing the use case actor/user) would need to make the first call.

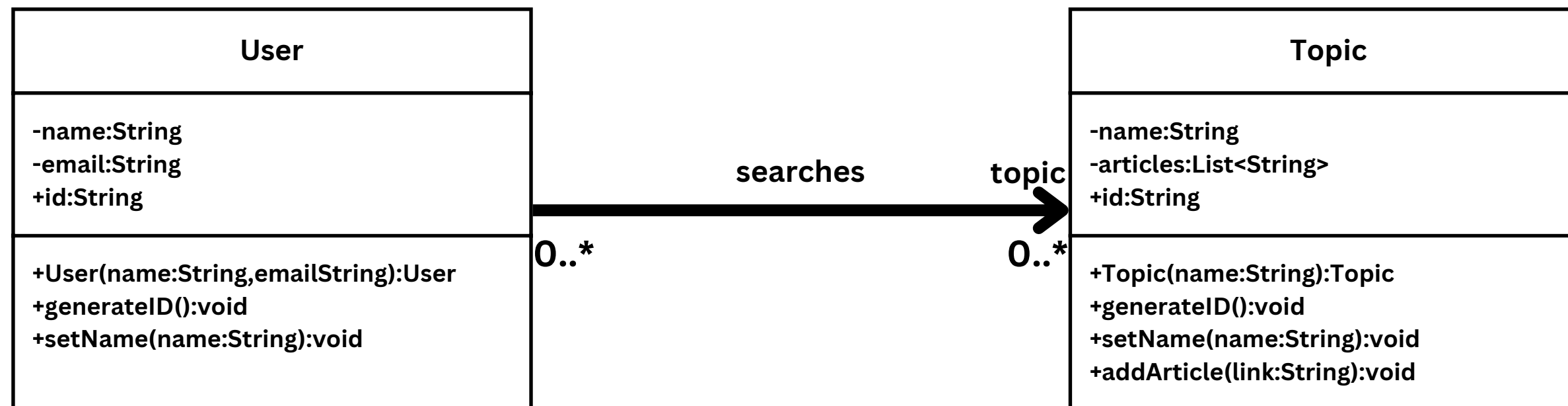
UML Sequence Diagram Modelling Steps

Step 1 -- Identifying main steps

Name	Gets balanced breakdown of articles
Purpose	User gets a clear, balanced breakdown of articles providing ‘for’ and ‘against’ perspectives on any news topic
Prerequisites	“Searches for a topic” use case completed.
Narrative	<u>1.The system fetches and displays a list of articles related to the selected topic.</u> <u>2. The system categorizes and presents a clear, balanced breakdown of articles, distinguishing between ‘for’ and ‘against’ perspectives on the topic.</u> <u>3. The system highlights articles using the “Views yielded articles highlighted in amber and red” use case.</u>
Outcome	<u>User views articles representing both sides of the argument for a balanced understanding of the topic.</u>
Alternative Narrative	The system does not find enough articles for one perspective of the topic.
Alternative Narrative Outcome	The system displays an error message instead of presenting articles from a single perspective.

Step 2 -- Identifying classes

Narrative	<u>1.The system fetches and displays a list of articles related to the selected topic.</u> <u>2. The system categorizes and presents a clear, balanced breakdown of articles, distinguishing between ‘for’ and ‘against’ perspectives on the topic.</u> <u>3. The system highlights articles using the “Views yielded articles highlighted in amber and red” use case.</u>
Outcome	<u>User views articles representing both sides of the argument for a balanced understanding of the topic.</u>

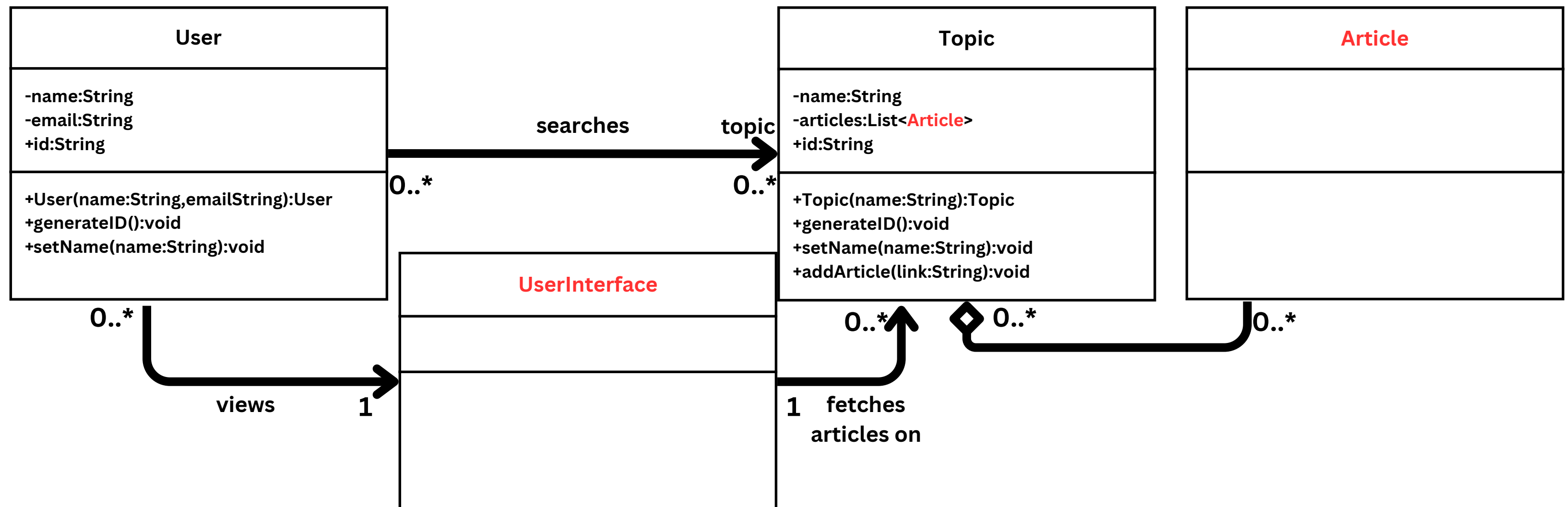


Steps:

1. Which classes are needed to fulfil the narrative and outcome?.
2. If there are needed classes missing in the class diagram, revise class diagram.

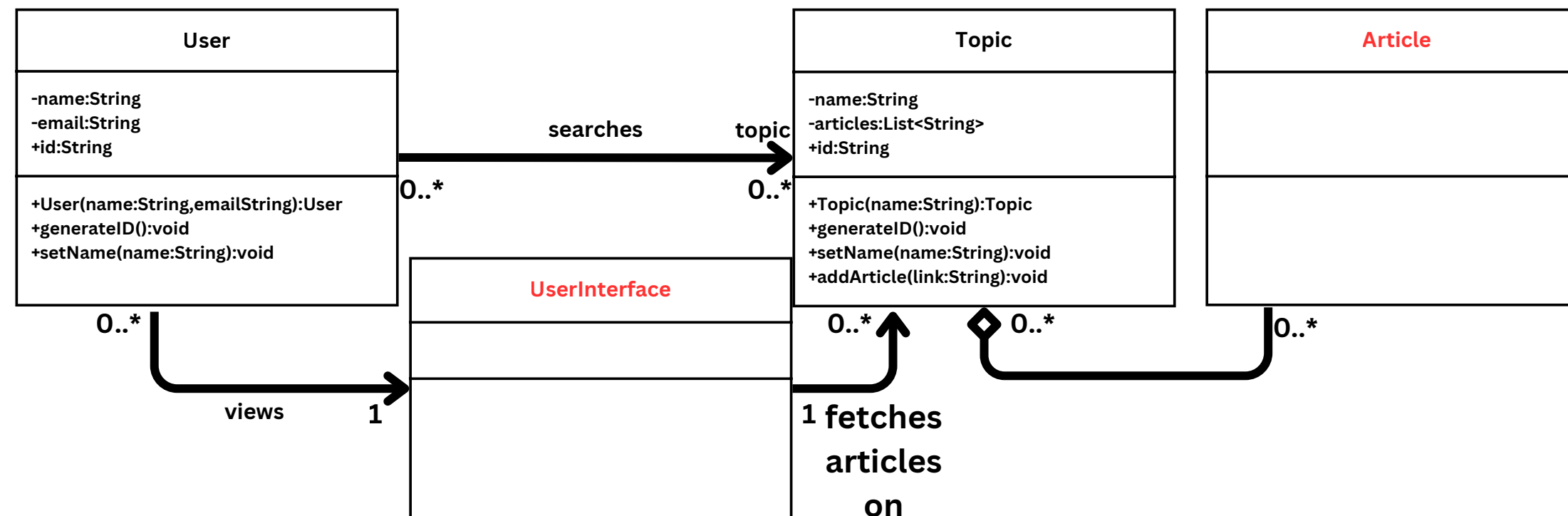
Step 2 -- Identifying classes

Narrative	<u>1.The system fetches and displays a list of articles related to the selected topic.</u> <u>2. The system categorizes and presents a clear, balanced breakdown of articles, distinguishing between ‘for’ and ‘against’ perspectives on the topic.</u> <u>3. The system highlights articles using the “Views yielded articles highlighted in amber and red” use case.</u>
Outcome	<u>User views articles representing both sides of the argument for a balanced understanding of the topic.</u>



Step 3 -- Identifying methods and associations

Narrative	<u>1.The system fetches and displays a list of articles related to the selected topic.</u> <u>2. The system categorizes and presents a clear, balanced breakdown of articles, distinguishing between ‘for’ and ‘against’ perspectives on the topic.</u> <u>3. The system highlights articles using the “Views yielded articles highlighted in amber and red” use case.</u>
Outcome	<u>User views articles representing both sides of the argument for a balanced understanding of the topic.</u>

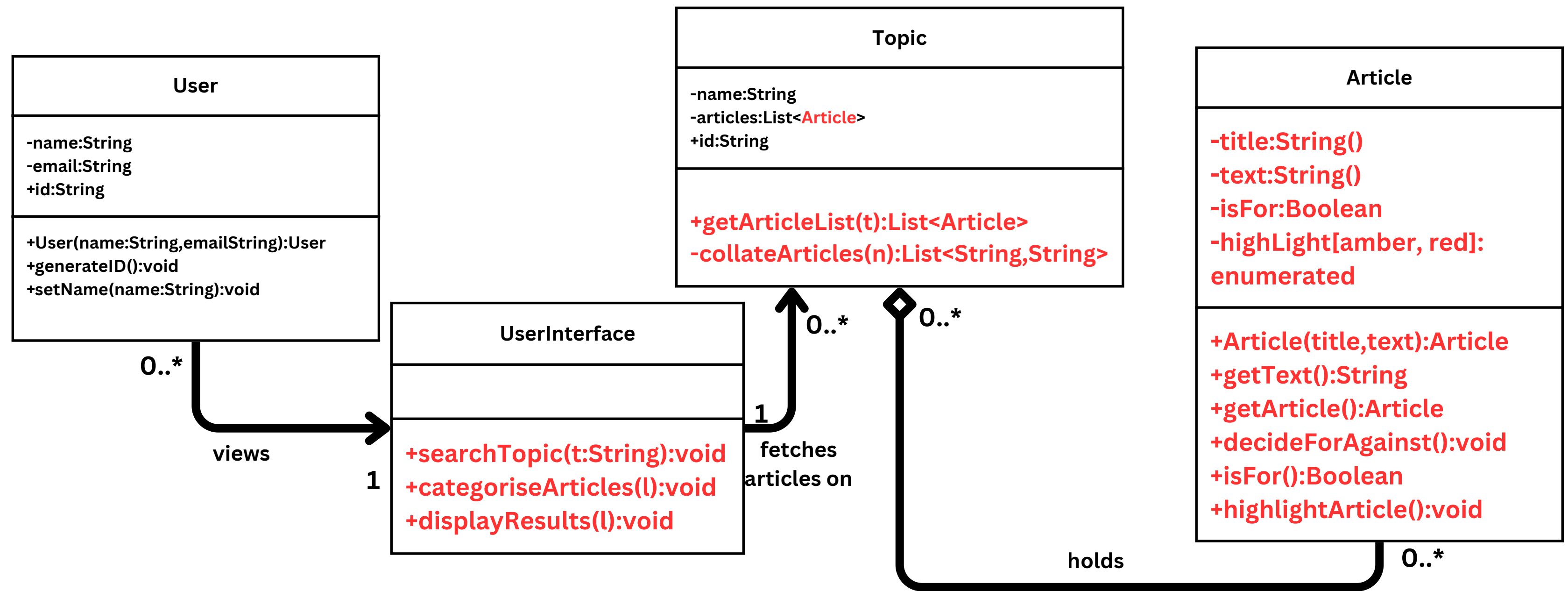


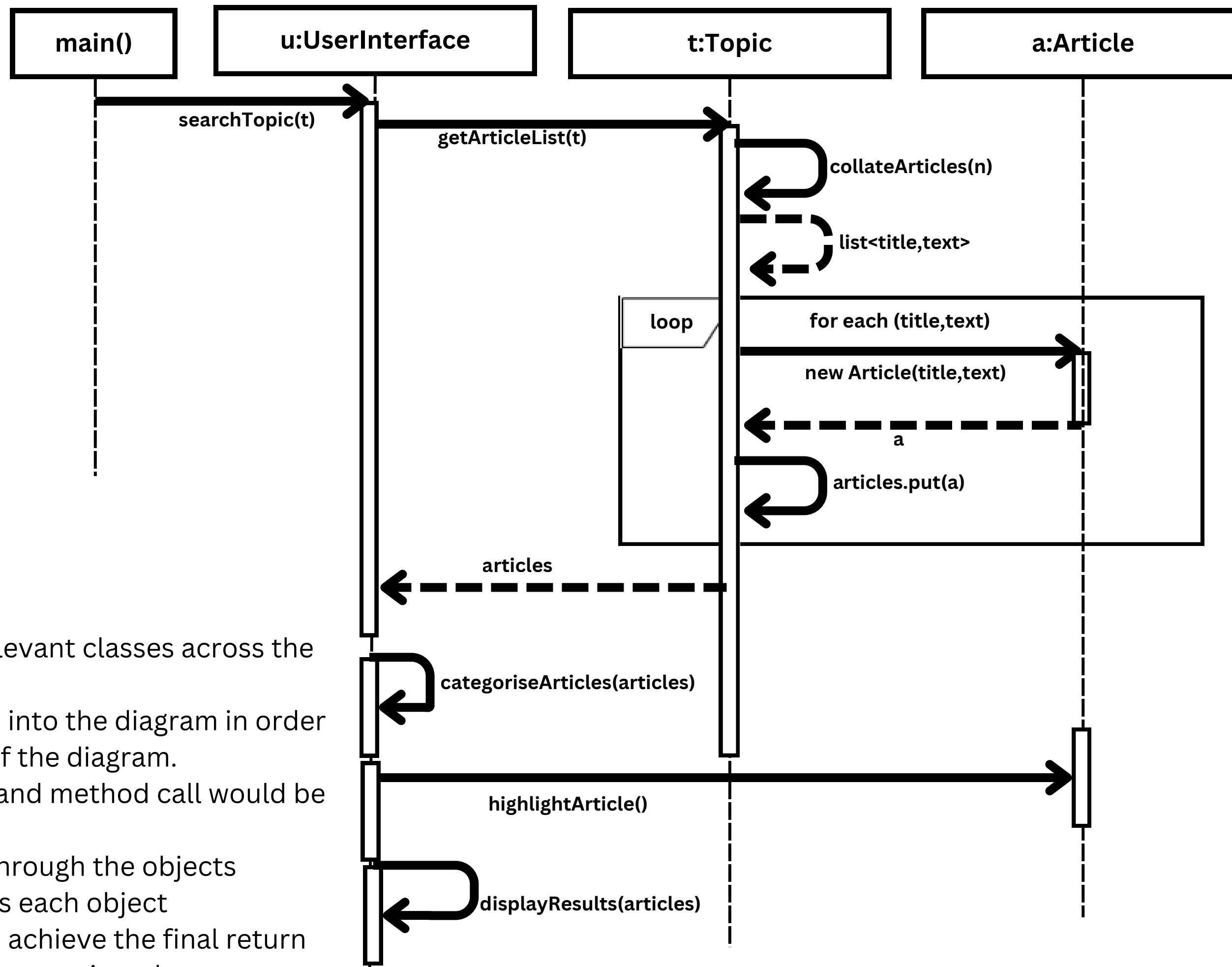
Steps:

1. Which methods and associations are needed to fulfil the narrative and outcome?
2. If there are needed methods and associations missing in the class diagram, revise class diagram.

Step 3 -- Identifying methods and associations

Narrative	<u>1.The system fetches and displays a list of articles related to the selected topic.</u> <u>2. The system categorizes and presents a clear, balanced breakdown of articles, distinguishing between ‘for’ and ‘against’ perspectives on the topic.</u> <u>3. The system highlights articles using the “Views yielded articles highlighted in amber and red” use case.</u>
Outcome	<u>User views articles representing both sides of the argument for a balanced understanding of the topic.</u>





Steps:

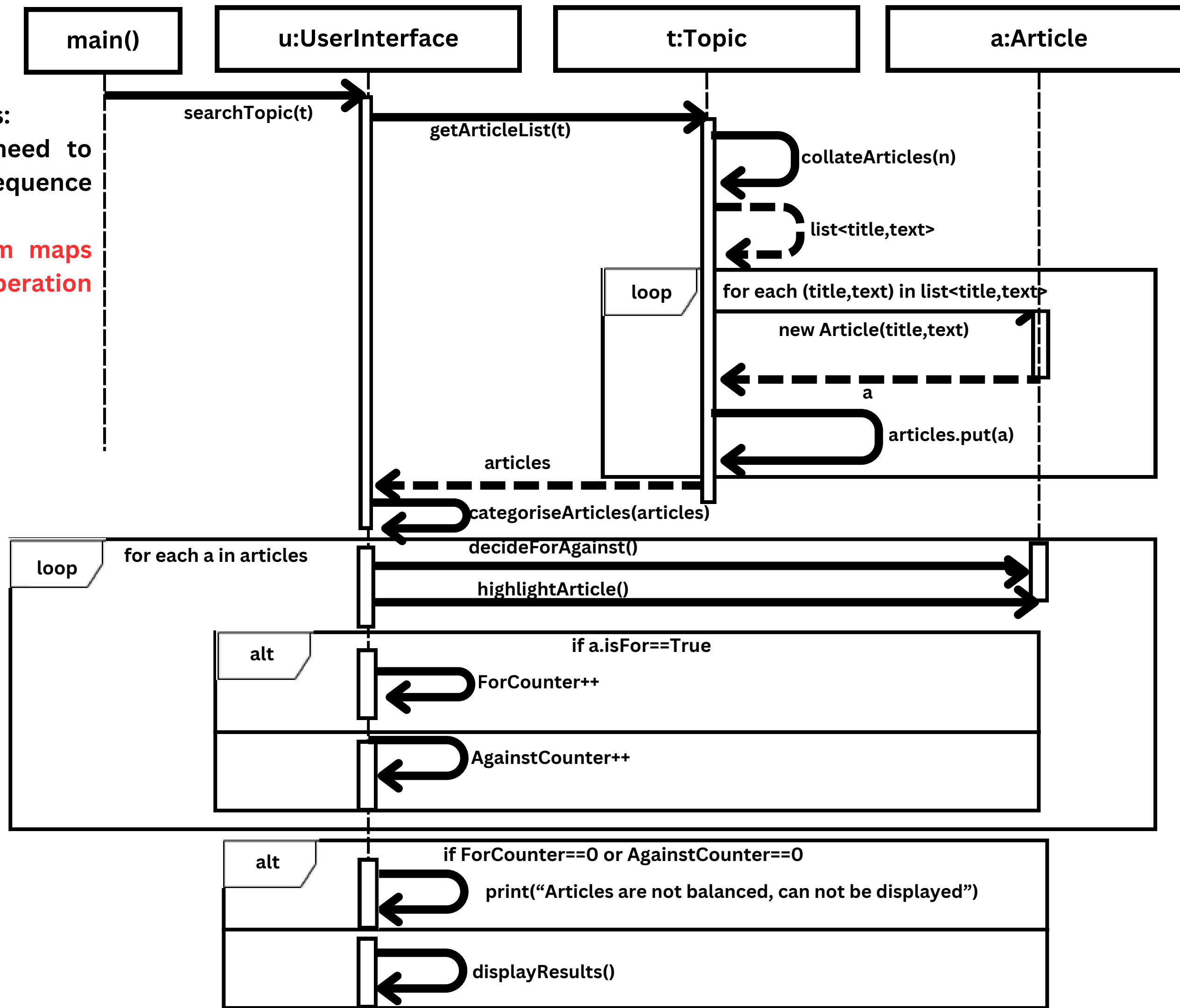
1. Put objects of the relevant classes across the top of the diagram.
2. Add the method calls into the diagram in order from top to bottom of the diagram.
3. Decide which object and method call would be the main trigger.
4. Follow the scenario through the objects deriving the messages each object receives/transmits to achieve the final return messages/store the appropriate data.

Step 4 -- Identifying alt/opt/loop envelopes

Name	Gets balanced breakdown of articles
Purpose	User gets a clear, balanced breakdown of articles providing ‘for’ and ‘against’ perspectives on any news topic
Prerequisites	“Searches for a topic” use case completed.
Narrative	1.The system fetches and displays a list of articles related to the selected topic. 2. The system categorizes and presents a clear, balanced breakdown of articles, distinguishing between ‘for’ and ‘against’ perspectives on the topic. 3. The system highlights articles using the “Views yielded articles highlighted in amber and red” use case.
Outcome	User views articles representing both sides of the argument for a balanced understanding of the topic.
Alternative Narrative	<u>The system does not find enough articles for one perspective of the topic.</u>
Alternative Narrative Outcome	<u>The system displays an error message instead of presenting articles from a single perspective.</u>

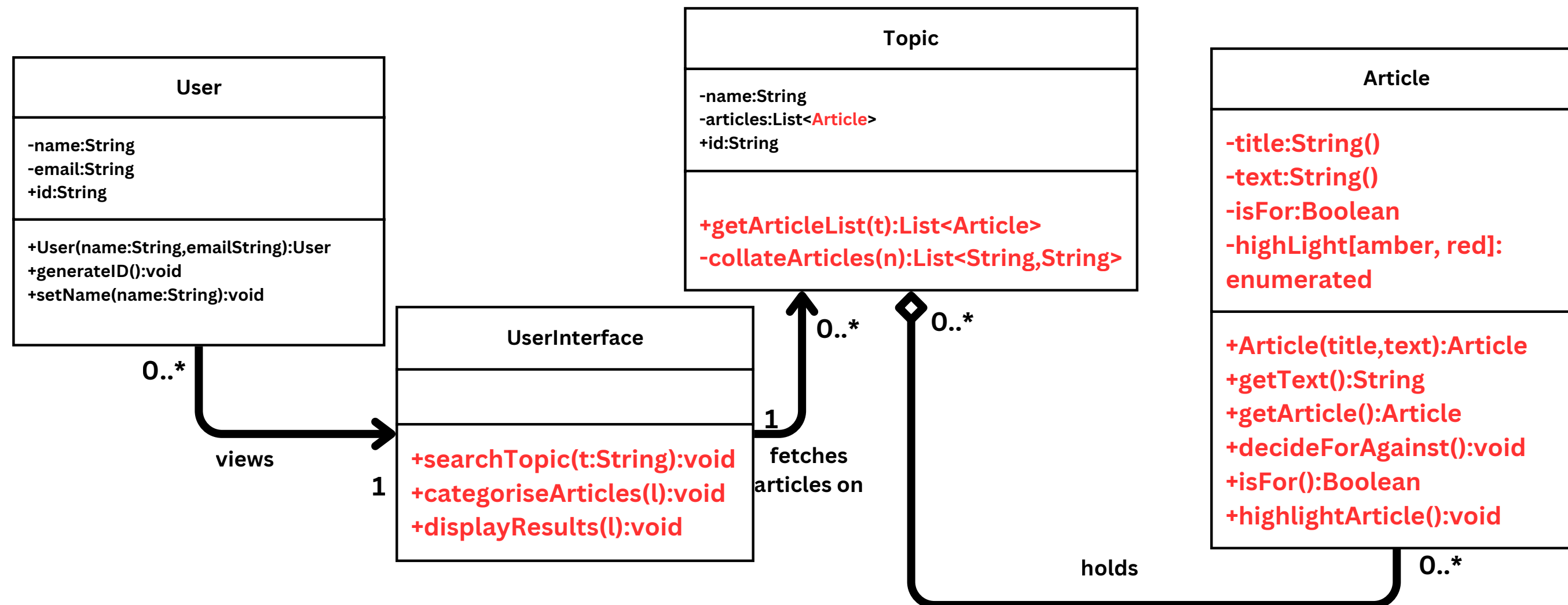
Tips for Sequence Diagrams:

- Not every class will need to have an object in the sequence diagram.
- Each sequence diagram maps to one use case or operation NOT the whole system.



Optional -- Improving class diagram

Narrative	<u>1.The system fetches and displays a list of articles related to the selected topic.</u> <u>2. The system categorizes and presents a clear, balanced breakdown of articles, distinguishing between ‘for’ and ‘against’ perspectives on the topic.</u> <u>3. The system highlights articles using the “Views yielded articles highlighted in amber and red” use case.</u>
Outcome	<u>User views articles representing both sides of the argument for a balanced understanding of the topic.</u>

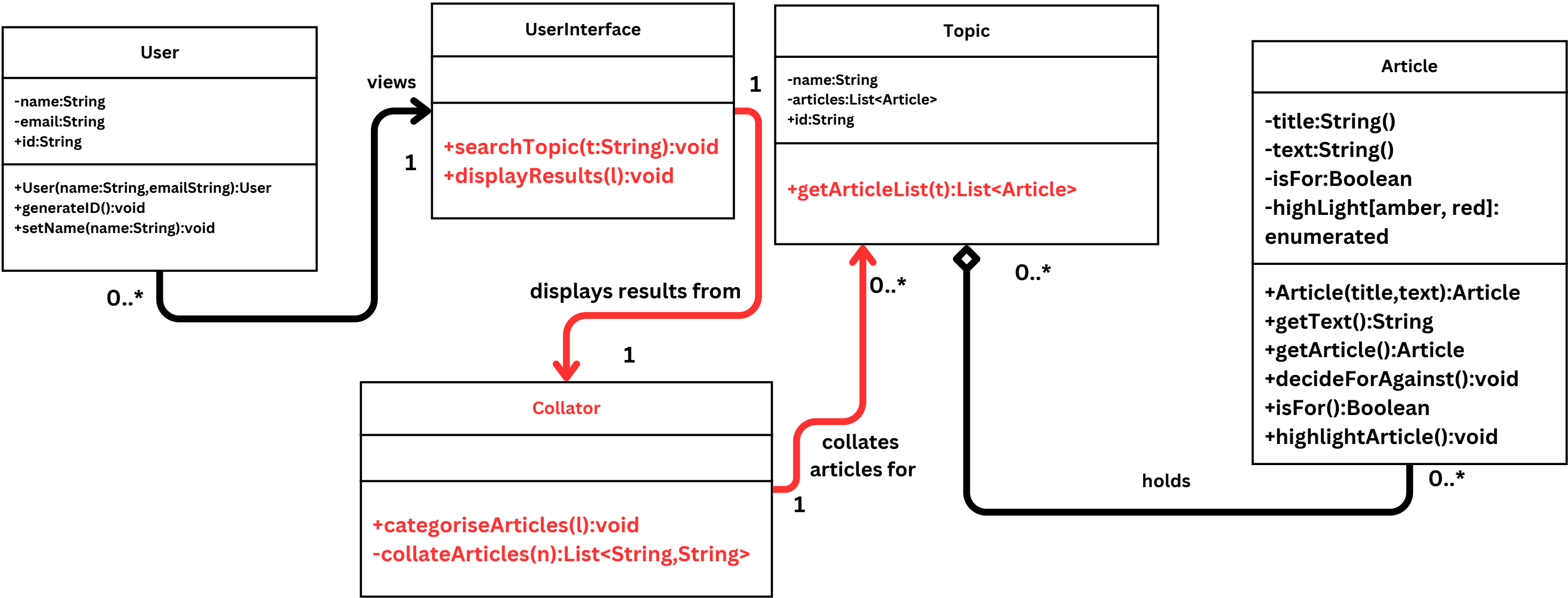


Steps:

1. How can overloading the classes with too many methods be avoided (i.e. make them more modular)?

Optional -- Improving class diagram

Narrative	<u>1.The system fetches and displays a list of articles related to the selected topic.</u> <u>2. The system categorizes and presents a clear, balanced breakdown of articles, distinguishing between ‘for’ and ‘against’ perspectives on the topic.</u> <u>3. The system highlights articles using the “Views yielded articles highlighted in amber and red” use case.</u>
Outcome	<u>User views articles representing both sides of the argument for a balanced understanding of the topic.</u>



From UserInterface class

```
public void searchTopic(String t) {  
    Topic topic = new Topic(t);  
    List<Article> articles = topic.getArticleList(t);  
    categoriseArticles(articles);  
    displayResults(articles);  
}  
  
public void categoriseArticles(List<Article> articles) {  
    int forCounter = 0;  
    int againstCounter = 0;  
  
    for (Article a : articles) {  
        a.decideForAgainst();  
        a.highlightArticle();  
  
        if (a.isFor()) {  
            forCounter++;  
        } else {  
            againstCounter++;  
        }  
    }  
  
    if (forCounter == 0 || againstCounter == 0) {  
        System.out.println("Articles are not balanced, cannot be displayed");  
        return;  
    }  
}
```



From Topic class

```
public class Topic {  
    private String name;  
    private List<Article> articles;  
    private String id;  
  
    public Topic(String name) {  
        this.name = name;  
        this.articles = new ArrayList<>();  
        this.id = generateId();  
    }  
}
```

From Topic class - continued

```
public List<Article> getArticleList(String t) {  
    Map<String, String> collatedArticles = collateArticles(t);  
  
    for (Map.Entry<String, String> entry : collatedArticles.entrySet()) {  
        articles.add(new Article(entry.getKey(), entry.getValue()));  
    }  
    return articles;  
}
```

```
private Map<String, String> collateArticles(String n) {  
    Map<String, String> articlesMap = new HashMap<>();  
  
    // Example static articles, replace with real data fetching logic  
    articlesMap.put("Title 1", "Content of article 1.");  
    articlesMap.put("Title 2", "Content of article 2.");  
  
    return articlesMap;  
}
```

```
}
```

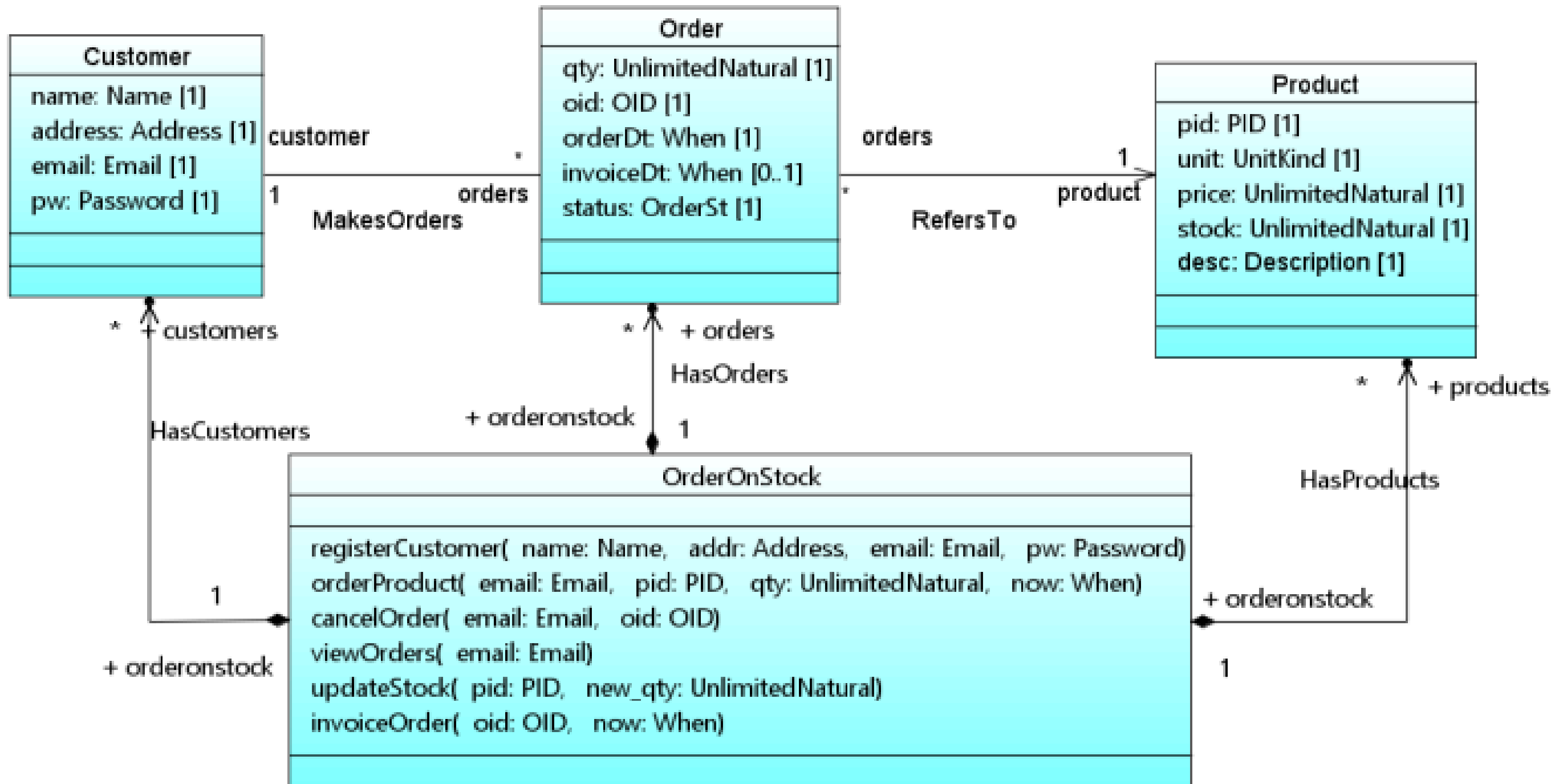
From Article class

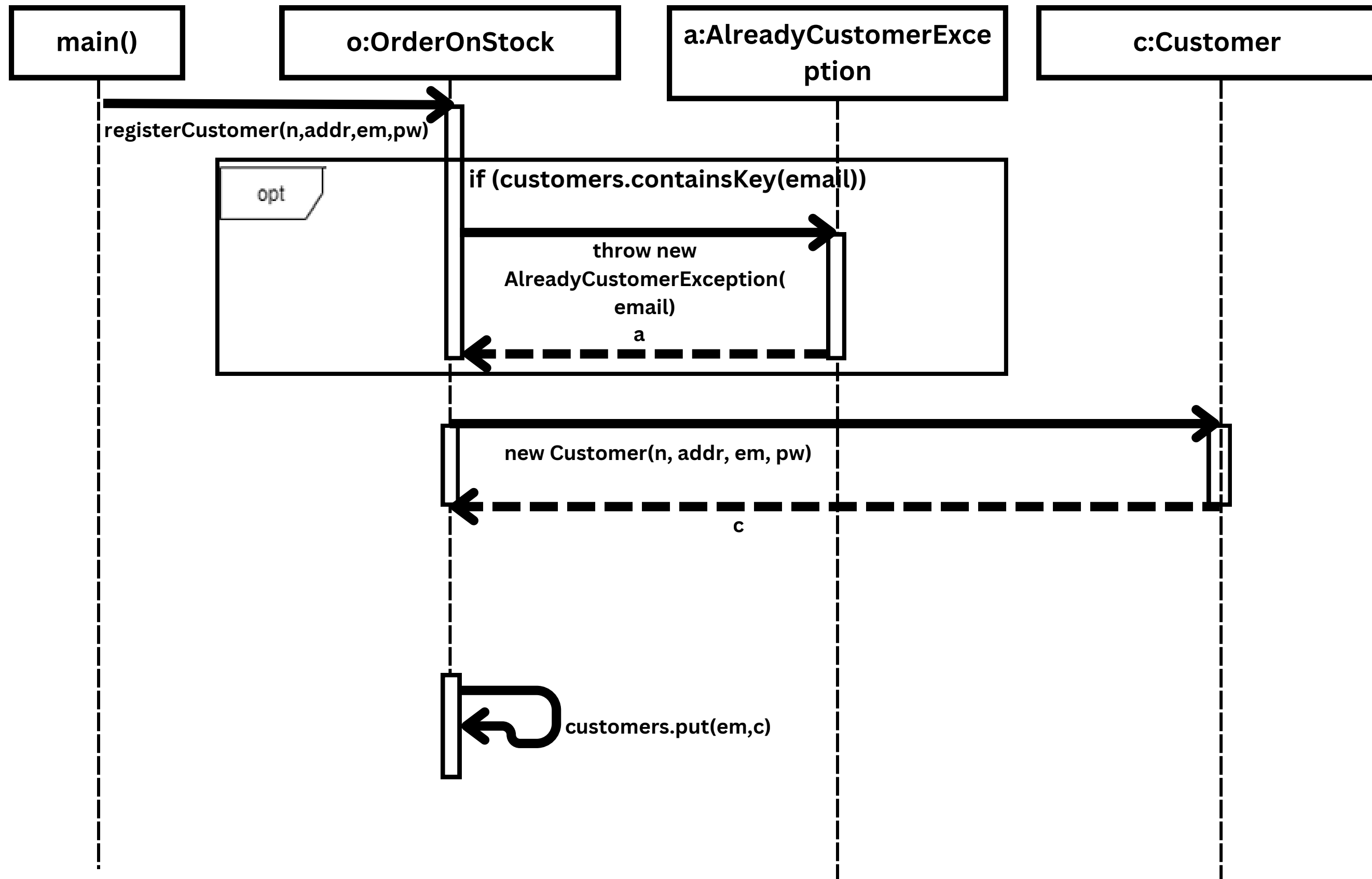
```
public class Article {  
    private String title;  
    private String text;  
    private boolean isFor;  
    private String highlight;  
  
    public Article(String title, String text) {  
        this.title = title;  
        this.text = text;  
        this.isFor = false; // Default value  
        this.highlight = "amber"; // Default highlight  
    }  
}
```

From Article class - continued

```
public boolean isFor() {  
    return isFor;  
}  
  
public void decideForAgainst() {  
    // Example Logic: articles with "support" are considered "for"  
    if (text.toLowerCase().contains("support")) {  
        isFor = true;  
    } else {  
        isFor = false;  
    }  
}  
  
public void highlightArticle() {  
    // Example Logic: highlight "red" if against, "amber" if for  
    if (isFor) {  
        highlight = "amber";  
    } else {  
        highlight = "red";  
    }  
}
```

Additional Example





```
public void registerCustomer(String name, String addr, String email, String pw)
{
    if (customers.containsKey(email)) {
        throw new AlreadyCustomerException(email);
    }

    Customer c = new Customer(name, addr, email, pw);

    customers.put(email, c);
}
```

What to include in D2?
Refer to Assessment Brief

AI Tools at BCU: Student Guidelines



AI tools Decision Tree

